

PegShield: A Solvency-as-a-Service Risk Oracle for Liquid-Staked Collateral on Solana

Anubhav Prasai

AI Youth Global Organization (AIYGO)

(Dated: May 15, 2026)

Abstract

PegShield is a Solana-native risk oracle for liquid staking token (LST) collateral. Unlike a price oracle, which estimates the exchange value of an asset, PegShield publishes an enforceable lending constraint: a live loan-to-value (LTV) recommendation and regime flag for a collateral market keyed by an LST identifier. This report formalizes PegShield as a solvency-as-a-service system: off-chain processes estimate peg-deviation dynamics from Pyth market prices and protocol-specific staking exchange rates, then commit a compact fixed-point risk state to a Solana program-derived address (PDA) consumed by lending protocols. The system targets LST de-pegging tail risk, where slow governance updates, stake-pool exit friction, and correlated liquidations can make static collateral factors insufficient. The current implementation comprises a Pyth/reference-rate bridge, an Ornstein-Uhlenbeck (OU) calibration engine, an Augmented Dickey-Fuller (ADF) and z-score regime detector, bounded liquidity and data-quality haircuts, a devnet Anchor oracle program, a threshold-attester update path, a TypeScript SDK, an operator CLI, and both TypeScript and Anchor reference consumers. The resulting primitive is not an assertion that a token is worth a given number of dollars; it is a machine-readable solvency boundary that downstream protocols can clamp, fallback, or use to halt new credit.

I. INTRODUCTION

Liquid staking resolves a capital-efficiency problem in proof-of-stake systems: staked assets need not remain economically inert. On Solana, receipt assets such as `mSOL`, `jitoSOL`, and `bSOL` represent claims on delegated SOL and can circulate through automated market makers, lending markets, vaults, and other DeFi contracts. Marinade describes `mSOL` as the liquid staking token received for staking SOL through its protocol, and Jito documents `JitoSOL` as the token associated with its stake-pool system [1, 2]. DeFiLlama and protocol dashboards show that liquid staking is a major component of the contemporary Solana DeFi surface [3].

This composability produces what this report calls the *Liquidity-Staking Paradox*. The same token that improves capital efficiency during normal conditions can become an unstable collateral object under stress. A stake-pool receipt token is redeemable against an underlying staking position, but immediate secondary-market liquidity is finite. When the receipt trades

below its canonical staking exchange rate, borrowers, liquidators, lenders, and arbitrageurs may all require exit liquidity in the same time interval. Static collateral factors, updated by governance or manual risk operations, are not designed to react at the speed of the borrow path.

PegShield addresses this mismatch by separating price discovery from collateral solvency control. The implemented question is:

Given current peg deviation, estimated mean reversion, volatility, stationarity, liquidity depth, and source quality, should a lending protocol extend new credit against this collateral, and at what LTV?

The answer is stored in a Solana PDA rather than in an off-chain dashboard. A lending protocol can read the account, verify ownership and freshness, and apply the suggested LTV or a stricter fallback policy inside its own borrowing logic.

II. PUBLIC ARTIFACTS

The implementation is accompanied by public, verifiable artifacts. These links are included because PegShield is intended to be evaluated as a deployed protocol surface, not only as a theoretical design.

TABLE I. Primary public artifacts and deployment references.

Artifact	Link
Devnet program	DMR3rXBh8RGrKyx1mxqFVTMbyfoiuu9iYHr6s6CW23ea
GitHub repository	github.com/Fianko-codes/PegShield
Live mSOL-v2 oracle state	7dtHBg6SyTykm1sDDvFPxoj7UJ12jqbFKSC5S8gpenGo
Updater authority	4kEmLqMqb3PGsmBC8brARQ5sKzUv37PjdSereu1yoNyc
Last documented devnet refresh	5cq1vDvEyfAyjsafW2TVAbGHAb8vyLHSoGrJWnQLLZy3K5KsmWBv2qfSwLnKoMKJJM8EcTuxGDwuAvz6teSmtMqy

III. REPOSITORY-DERIVED IMPLEMENTATION SCOPE

This report is derived from the repository implementation rather than from a proposed roadmap. Table II summarizes the inspected components.

TABLE II. Implementation surfaces used for this report.

Surface	Role
Bridge	Pyth/reference-rate ingestion and peg-deviation construction.
Core engine	OU calibration, ADF/z-score regime detection, LTV haircuts.
Updater	Fixed-point payload submission and consumer comparison scripts.
Risk oracle	Anchor oracle, risk state, attesters, disputes, slashing records.
Mock lender	Anchor borrow-gate reference consumer.
SDK	Decoder, PDA derivation, owner verification, freshness and LTV guards.
CLI	Operator commands, policy proof, multi-attester proof, scenario lab.
Lending example	External TypeScript lending consumer.
Scripts	Artifact regeneration, validation, and oracle-cycle wiring.
Artifacts/simulation	Oracle snapshots and stress-scenario bundles.

The current deployment state is explicitly pre-production. The repository documents a devnet Anchor program ID `DMR3rXBh8RGrKyx1mxqFVTMbyfoiuu9iYHr6s6CW23ea`, a devnet `mSOL-v2 RiskState` PDA, and an updater authority. The code supports `mSOL-v2`, `jitoSOL-v1`, and `bSOL-v1`; not every supported asset has a populated PDA in the committed artifacts. Security documentation also states that the program is not audited, not mainnet production infrastructure, and remains upgrade-mutable.

IV. SYSTEM ARCHITECTURE

A. Data Plane

Let a denote an LST market and let s denote SOL. The bridge reads two Pyth market prices, P_t^a and P_t^s , and a protocol-specific reference exchange rate R_t^a . The implemented supported sources are:

- Marinade API for `mSOL-v2`;
- Jito stake-pool statistics for `jitoSOL-v1`;
- BlazeStake stake-pool account data over Solana RPC for `bSOL-v1`.

The canonical risk signal is not the raw USD premium. It is the peg deviation

$$X_t^a = \frac{P_t^a / P_t^s}{R_t^a} - 1. \quad (1)$$

This distinction is essential because an LST can trade above SOL in USD terms simply because the staking exchange rate has accrued rewards. Peg risk begins when the market ratio trades below the protocol redemption ratio.

The bridge rejects malformed price data, carries Pyth confidence values, records whether historical data were live or cache-derived, and marks reference-rate fallbacks. The engine then treats degraded input quality as a separate source of risk rather than as a neutral missing value.

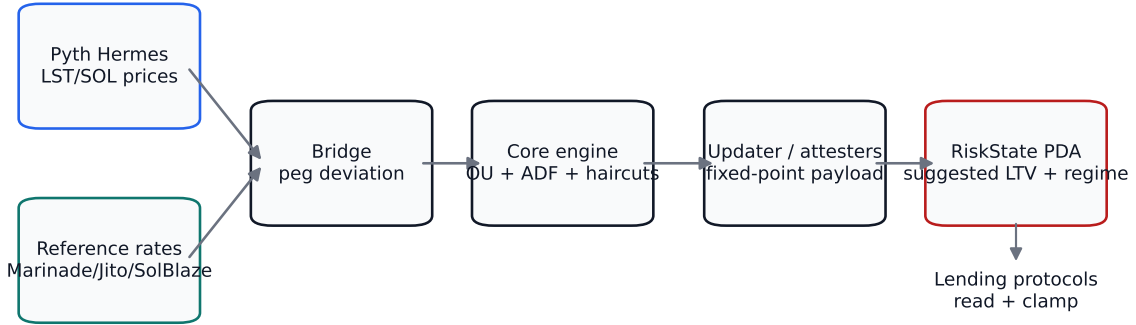


FIG. 1. Implemented PegShield data plane. Pyth market feeds and protocol reference rates are normalized by the bridge, calibrated by the off-chain engine, encoded by the updater or attester path, and committed to a **RiskState** PDA consumed by lending protocols.

B. Risk-State Commitment

The on-chain oracle account is an Anchor account **RiskState**, derived by

$$\text{PDA}_a = \text{findProgramAddress}(["\text{risk}", \text{bytes}(a)], \text{program_id}). \quad (2)$$

The implemented fields are shown in Table III.

The oracle program performs no floating-point computation on-chain. Its role is to validate account identity, signer authority, update mode, basic numeric bounds, timing constraints, and threshold-attester state transitions. In single-attester mode, `update_risk_state`

TABLE III. RiskState account fields. Floating values are stored in fixed point.

Field	Type	Meaning
lst_id	String	LST market identifier, max 16 bytes.
theta_scaled	i64	OU mean-reversion speed $\theta \times 10^6$.
sigma_scaled	i64	OU volatility $\sigma \times 10^6$.
regime_flag	u8	0 = normal, 1 = critical.
suggested_ltv_bps	u16	Suggested LTV in basis points.
z_score_scaled	i64	Current z-score $z \times 10^6$.
slot	u64	Slot of last accepted update.
timestamp	i64	Unix timestamp of last accepted update.
authority	Pubkey	Single-attester writer in compatibility mode.
last_updater	Pubkey	Most recent submitter/confirming signer.
update_mode	u8	0 = single-attester, 1 = multi-attester.
attester_registry	Pubkey	Registry PDA for multi-attester mode.

requires the stored **authority** signer, rejects multi-attester accounts, checks that the supplied **lst_id** matches the PDA state, enforces $\theta_{\text{scaled}} \geq 0$, $\sigma_{\text{scaled}} > 0$, and $0 \leq \text{LTV}_{\text{bps}} \leq 10000$, and rate-limits updates to at least 30 seconds apart.

V. STATISTICAL MODEL AND SOLVENCY INDEX

A. OU Calibration

The engine fits an Ornstein-Uhlenbeck model to the peg-deviation series in Eq. (1). In continuous time,

$$dX_t = \theta(\mu - X_t)dt + \sigma dW_t, \quad (3)$$

where $\theta > 0$ denotes mean-reversion speed, μ the local equilibrium deviation, $\sigma > 0$ instantaneous volatility, and W_t a Brownian motion. The implementation estimates the discrete regression

$$\Delta X_i = \alpha + \beta X_{i-1} + \varepsilon_i, \quad (4)$$

with sampling interval Δt in days. It then maps

$$\hat{\theta} = -\frac{\hat{\beta}}{\Delta t}, \quad \hat{\mu} = \frac{\hat{\alpha}}{-\hat{\beta}}, \quad \hat{\sigma} = \frac{\text{sd}(\hat{\varepsilon})}{\sqrt{\Delta t}}, \quad (5)$$

with positive lower bounds used to avoid degenerate zero estimates. The baseline parameters $(\bar{\theta}, \bar{\sigma})$ are medians over rolling windows when enough history exists, otherwise the current-window estimate is used.

B. Regime Detection

The current z-score is

$$z_t = \frac{X_t - \bar{X}_{[t-n, t]}}{s_{[t-n, t]}}. \quad (6)$$

The engine applies an ADF test to the rolling deviation window and declares

$$C_t = \mathbf{1}\{|z_t| > 2.5\} \mathbf{1}\{p_{\text{ADF}, t} \geq 0.05\}. \quad (7)$$

Thus **CRITICAL** requires both an extreme standardized deviation and failure of the stationarity test. This avoids treating every ordinary variance realization as a structural break.

C. Liquidity, Validator Health, and Peg Stability

LST solvency depends on more than price. Let L_t denote effective exit liquidity available to liquidators and arbitrageurs, and let $H_t \in [0, 1]$ denote a latent validator/stake-pool health variable capturing delegation performance, withdrawal friction, and reference-rate reliability. Peg stability can be modeled as a jump-diffusion extension of Eq. (3):

$$dX_t = \theta_t(\mu_t - X_t) dt + \sigma_t dW_t - \kappa_L dN_t^L - \kappa_H dN_t^H, \quad (8)$$

where liquidity shocks and validator-health shocks have intensities

$$\lambda_t^L = \lambda_0^L \exp\left(\beta_L \left(1 - \frac{L_t}{L_t^*}\right)_+\right), \quad (9)$$

$$\lambda_t^H = \lambda_0^H \exp(\beta_H(1 - H_t)_+). \quad (10)$$

The current code does not ingest validator-performance telemetry as a first-class field. It uses the canonical staking exchange rate as the accounting reference and applies data-quality haircuts when reference-rate sources are fallback-derived. The code does, however, accept generic liquidity metrics and maps them into bounded haircuts.

D. Implemented LTV Functional

The implemented base constants are

$$\ell_0 = 0.80, \quad \ell_{\min} = 0.40, \quad \ell_{\max} = 0.80. \quad (11)$$

The statistical LTV is

$$\ell_t^{\text{stat}} = \begin{cases} \ell_{\min}, & C_t = 1, \\ \text{clip} \left(\ell_0 \frac{\hat{\theta}_t}{\bar{\theta}_t} \frac{\bar{\sigma}_t}{\hat{\sigma}_t}, \ell_{\min}, \ell_{\max} \right), & C_t = 0. \end{cases} \quad (12)$$

Liquidity and data-quality risk maps return bounded haircuts

$$h_t^L \in [0, 0.30], \quad h_t^Q \in [0, 0.15]. \quad (13)$$

The published LTV is

$$\ell_t = \text{clip} \left(\ell_t^{\text{stat}} - h_t^L - h_t^Q, \ell_{\min}, \ell_{\max} \right). \quad (14)$$

The solvency index is the normalized enforced collateral factor

$$\Psi_t = \frac{\ell_t}{\ell_0} \in \left[\frac{1}{2}, 1 \right], \quad (15)$$

where $\Psi_t = 1$ means the model permits the base collateral factor and $\Psi_t = 1/2$ means the system has reached the configured 40% floor. For a lending protocol with collateral value V_t and requested debt B_t , PegShield’s reference credit constraint is

$$B_t \leq V_t \ell_t. \quad (16)$$

The protocol remains free to apply a stricter cap, fallback, pause, or liquidation policy.

VI. POLICY PROOFS AND STRESS EVIDENCE

The repository’s `policy-proof` command is a deterministic artifact-level proof, not a formal theorem prover. It loads the committed stress bundle and oracle snapshot, constructs a collateral scenario, and compares a fixed 80% table against the dynamic PegShield LTV. In the June 2022 stETH/ETH replay, the model emitted its first critical signal on 2022-06-09 at approximately -3.40% peg deviation, while the fixed-LTV baseline first exhibited shortfall on 2022-06-13. The committed case-study artifact reports a peak fixed-80% shortfall of

\$51,946.38, a peak PegShield shortfall of \$0.00 in the modeled 100-unit position, and a peak LTV cut of 40 percentage points. These results validate early collateral tightening in the replay; they do not prove zero market-wide bad debt, production liquidator performance, or universal calibration across all LSTs.

The scenario lab currently contains one historical replay and multiple synthetic paths: liquidity vacuum, reflexive bank run, slow grind depeg, false-positive wick, flash-crash repricing, SVB-weekend bank run, stablecoin redemption spiral, and restaking bridge contagion. The synthetic scenarios are useful for regression testing the policy surface but should not be cited as empirical market measurements.

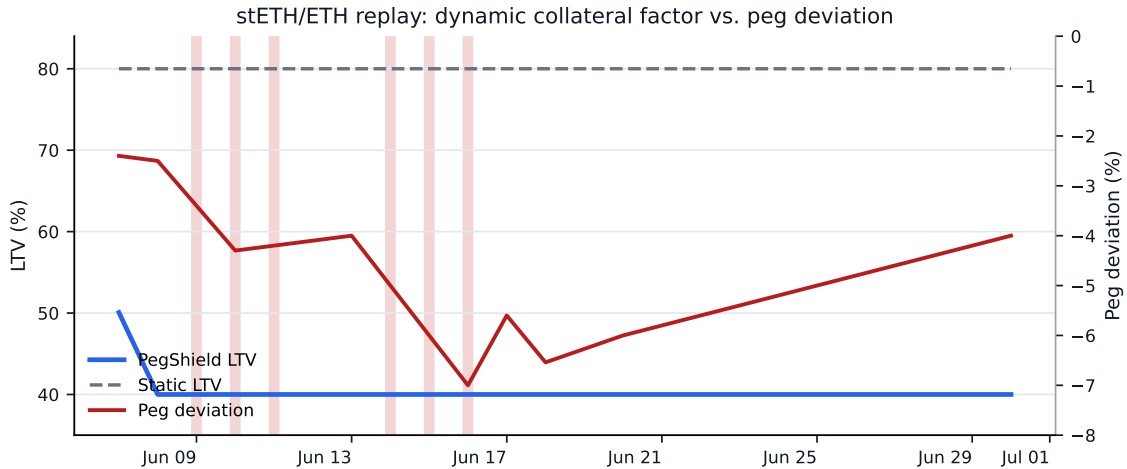


FIG. 2. Historical stETH/ETH replay. The dynamic LTV falls from the static 80% baseline to the 40% floor as peg deviation deteriorates. In terms of the solvency index, this corresponds to a movement from $\Psi_t = 1$ to $\Psi_t = 0.5$.

VII. ATTESTER CYCLES

A. Implemented Account Model

The multi-attester path preserves the consumer interface: protocols still read the same `RiskState` PDA. The trust model behind writes changes from a single authority to a bonded registry and pending-update cycle.

The implemented cycle is:

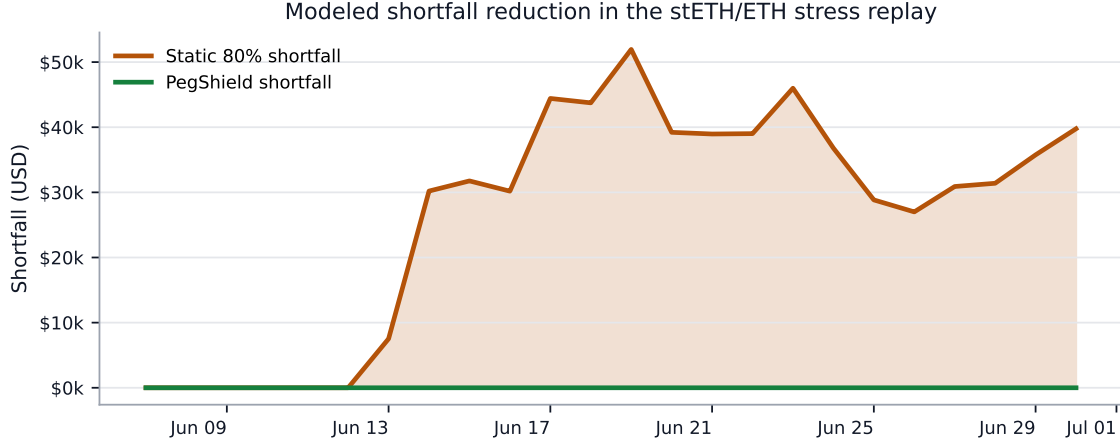


FIG. 3. Modeled shortfall comparison for the same replay. The static 80% policy develops positive shortfall after the depeg widens; the PegShield policy has already tightened to the floor in this scenario-scale collateral model.

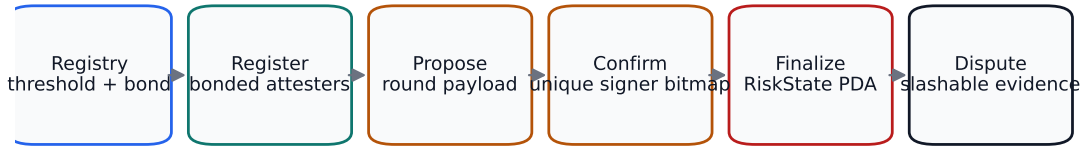
TABLE IV. Implemented threshold-attester constants and accounts.

Item	Implemented behavior
MAX_ATTESTERS	5 fixed registry slots.
MIN_BOND	1 SOL minimum registration bond.
PENDING_TTL	300 seconds.
DISPUTE_WINDOW	3600 seconds after finalization.
RESOLUTION_DEADLINE	24 hours after dispute filing.
SLASH_PERCENT	50% of attester bond.
DISPUTER_REWARD	50% of slashed amount.
AttesterRegistry	Admin, threshold, minimum bond, slash destination, and attester entries.
PendingUpdate	Round payload, proposer, expiration, bitmap confirmations, finalization metadata.
DisputeRecord	Evidence hash, disputed attester, deadline, slash outcome, and copied disputed parameters.

1. `initialize_registry(threshold,min_bond)` creates the registry PDA and sets admin, threshold, minimum bond, and slash destination.
2. `register_attester(bond_amount)` transfers lamports from the attester to the registry PDA and records an active attester slot.
3. `enable_multi_attester(lst_id)` switches a `RiskState` from single-attester to multi-attester mode if the registry can satisfy its threshold.

4. `propose_update` creates a pending round. The proposer must be active and auto-confirms via a bitmap bit indexed by registry position.
5. `confirm_update` requires a registered, unique, not-yet-confirming attester and a non-expired pending update. Once confirmation count reaches threshold, the risk parameters are copied into `RiskState`.
6. `cancel_expired` can close an expired unfinalized pending update and refund rent to the proposer.
7. `dispute_update` can be filed within the finalization dispute window against an attester who participated in the round.
8. `resolve_dispute` is admin-gated. If slashing is selected, 50% of the attester bond is deducted; half of the slashed amount goes to the disputer and half to the configured slash destination.

The current program authenticates attesters by Solana transaction signer identity and registry membership. It does not yet implement an on-chain Ed25519 signature aggregation verifier over an off-chain canonical payload. The CLI's `multi-attester-proof` is therefore a deterministic readiness proof for the implemented state machine, not evidence of a live independent production committee.



Consumers continue to read the same `RiskState PDA`; only the write trust model changes.

FIG. 4. Threshold-attester lifecycle implemented in the Anchor program. The consumer PDA remains stable while the write path moves from a single authority to bonded proposal, confirmation, finalization, and dispute states.

VIII. CONSUMER REFERENCE IMPLEMENTATION

A. SDK Policy

The TypeScript SDK exposes `fetchRiskState`, `decodeRiskState`, PDA derivation, and guards. `fetchRiskState` rejects missing accounts and rejects accounts not owned by the expected PegShield program. `isStale` treats timestamp zero as stale and defaults to a 600-second maximum age. `isCritical` returns true when `regimeFlag == 1`. The reference policy is:

$$\text{safeLtv}(S_t) = \begin{cases} \ell_{\text{fallback}}, & \text{stale}(S_t), \\ \ell_{\text{fallback}}, & S_t.\text{regimeFlag} = 1, \\ \min(\max(S_t.\text{suggestedLtv}, 0), \ell_{\text{protocolMax}}), & \text{otherwise.} \end{cases} \quad (17)$$

The default fallback in the SDK is 40%, while example consumers commonly cap the maximum at 80% or a protocol-selected ceiling.

B. Anchor Borrow Gate

The `mock-lender` program is a reference enforcement module rather than a toy off-chain script. It stores a `BorrowPolicy` PDA with market admin, LST ID, maximum LTV, fallback LTV, maximum oracle age, critical-regime behavior, pause flag, and update timestamp. `assess_borrow` reads the PegShield `RiskState` through `seeds::program = risk_oracle::ID`, validates matching LST IDs, then records a `BorrowDecision` PDA with the applied LTV, oracle age, reason code, and allow/reject outcome. Its implemented decision logic is:

$$\ell_t^{\text{applied}} = \begin{cases} 0, & \text{policy paused,} \\ \ell_{\text{fallback}}, & \text{oracle stale,} \\ 0, & \text{critical and halt-on-critical,} \\ \ell_{\text{fallback}}, & \text{critical and not halt-on-critical,} \\ \min(\ell_t^{\text{oracle}}, \ell_{\text{policyMax}}), & \text{otherwise.} \end{cases} \quad (18)$$

The borrow is allowed only if requested debt is no greater than $V_t \ell_t^{\text{applied}}$.

IX. SECURITY AND MANIPULATION ANALYSIS

A. Oracle Manipulation Surface

PegShield is less sensitive to ordinary USD-price drift than a naive LST/USD spread because it normalizes the LST/SOL market ratio by a protocol reference rate. A manipulator must therefore influence the relative LST/SOL market price, the Pyth feed, or the canonical reference-rate source. The current code includes several defensive properties:

- on-chain writers cannot submit negative θ , non-positive σ , or LTV outside $[0, 10000]$ basis points;
- single-attester writes require the stored authority and are rate-limited;
- multi-attester writes require active bonded registry members and threshold confirmation before finalization;
- source degradation is represented as a data-quality haircut rather than ignored;
- missing liquidity metrics are penalized by the data-quality component;
- SDK consumers verify owner, decode layout, freshness, and critical regime before using the feed;
- consumer policy is asymmetric: stale, missing, or critical data lowers LTV or halts borrowing.

These mechanisms do not make the oracle manipulation-proof. The repository security document explicitly lists dependency on Pyth honesty/timeliness, reference-rate integrity, mutable upgrade authority, unaudited code, and pre-production independent attester operations as limitations. The current implementation also does not cross-check Pyth against a secondary oracle and does not recompute OU/ADF logic on-chain.

B. Composability

PegShield is composable because its primary output is a single PDA per LST, not a bespoke integration process. A protocol can consume it in three ways:

1. off-chain or client-side via `@pegshield/sdk`;
2. directly in an Anchor program by deriving `["risk", lst_id]` under the PegShield program ID and decoding the account;
3. through the reference borrow gate pattern, which records auditable borrow decisions in a market-owned policy account.

The design intentionally does not force a lender to surrender final risk authority. Protocols retain their own maximum LTVs, fallbacks, market pauses, and critical-regime policies. PegShield supplies the time-varying solvency constraint.

X. OPERATIONAL FLOW

The seven-step oracle cycle encoded in `scripts/run_oracle_cycle.sh` is:

1. run engine unit tests;
2. fetch live Pyth prices and the relevant reference rate;
3. run the statistical engine;
4. submit a devnet update;
5. read the PDA back through the SDK;
6. replay the stETH stress scenario;
7. sync oracle and simulation artifacts.

The stricter `scripts/validate_protocol.sh` adds security scanning for tracked secret-like files, engine tests, SDK tests, CLI tests, case-study generation, SDK and CLI builds, the policy proof, the multi-attester proof, scenario lab output, Rust `cargo check`, mock-lender unit tests, oracle-cycle dry-run wiring, and artifact-presence checks.

The updater converts floats to fixed point using scale 10^6 and converts `suggested_ltv` to basis points:

```
theta_scaled = round(theta * 1_000_000)
sigma_scaled = round(sigma * 1_000_000)
z_score_scaled = round(z_score * 1_000_000)
suggested_ltv_bps = round(clamp(suggested_ltv, 0, 1) * 10_000)
```

This serialization boundary is the interface between stochastic off-chain computation and deterministic on-chain state.

XI. CURRENT ARTIFACT SNAPSHOT

The committed snapshots dated 2026-04-24 report **NORMAL** regime for the three configured LST identifiers and `suggested_ltv = 0.7325`. The reduction from the statistical 0.80 LTV is explained by a 0.0675 data-quality haircut caused by fallback history and missing liquidity-depth metrics. The snapshots mark `analytics_status` as `withheld_fallback_history`; therefore they should be interpreted as committed operational artifacts, not as a fresh production calibration.

XII. LIMITATIONS AND RESEARCH AGENDA

The implementation is sufficient to define a concrete solvency-as-a-service primitive, but several limitations are explicit:

- devnet-only deployment status;
- no completed independent production attester set;
- no external audit;
- no secondary oracle cross-check;
- no on-chain recomputation of statistical model outputs;
- latent validator health is not currently ingested as a separate telemetry stream;
- synthetic scenarios are regression tests, not empirical guarantees;

- the LTV mapping constants require production calibration before mainnet use.

The natural research agenda is to add secondary price-source checks, validator and stake-pool health telemetry, formal verification of LTV clamp invariants, production multi-attester operations, immutable or multisig-controlled program upgrades, and venue-specific liquidity-depth calibration for each supported LST.

XIII. CONCLUSION

PegShield implements a risk-oracle architecture for LST collateral in which solvency constraints, rather than asset prices, are the primary output. The codebase establishes an end-to-end path from Pyth/reference-rate ingestion to OU/ADF risk estimation, fixed-point Solana state, attester-mediated updates, and lending-protocol borrow gates. The resulting standard is operationally simple for consumers: read one PDA, verify ownership and freshness, respect critical regime, clamp to local policy, and fallback conservatively. The scientific contribution is the explicit conversion of peg-deviation tail risk into an enforceable on-chain credit boundary Ψ_t , making LST collateral management a continuous solvency service rather than a static governance parameter.

-
- [1] Marinade Finance, What is mSOL?, <https://docs.marinade.finance/marinade-protocol/protocol-overview/marinade-liquid/what-is-msol> (2026), accessed 2026-05-15.
 - [2] Jito Foundation, JitoSOL Liquid Staking, <https://www.jito.network/docs/jitosol/jitosol-liquid-staking/> (2026), accessed 2026-05-15.
 - [3] DeFiLlama, Jito Liquid Staking, <https://defillama.com/protocol/jito-liquid-staking> (2026), accessed 2026-05-15.